

Agosto 14, 2026

Patrones de Software

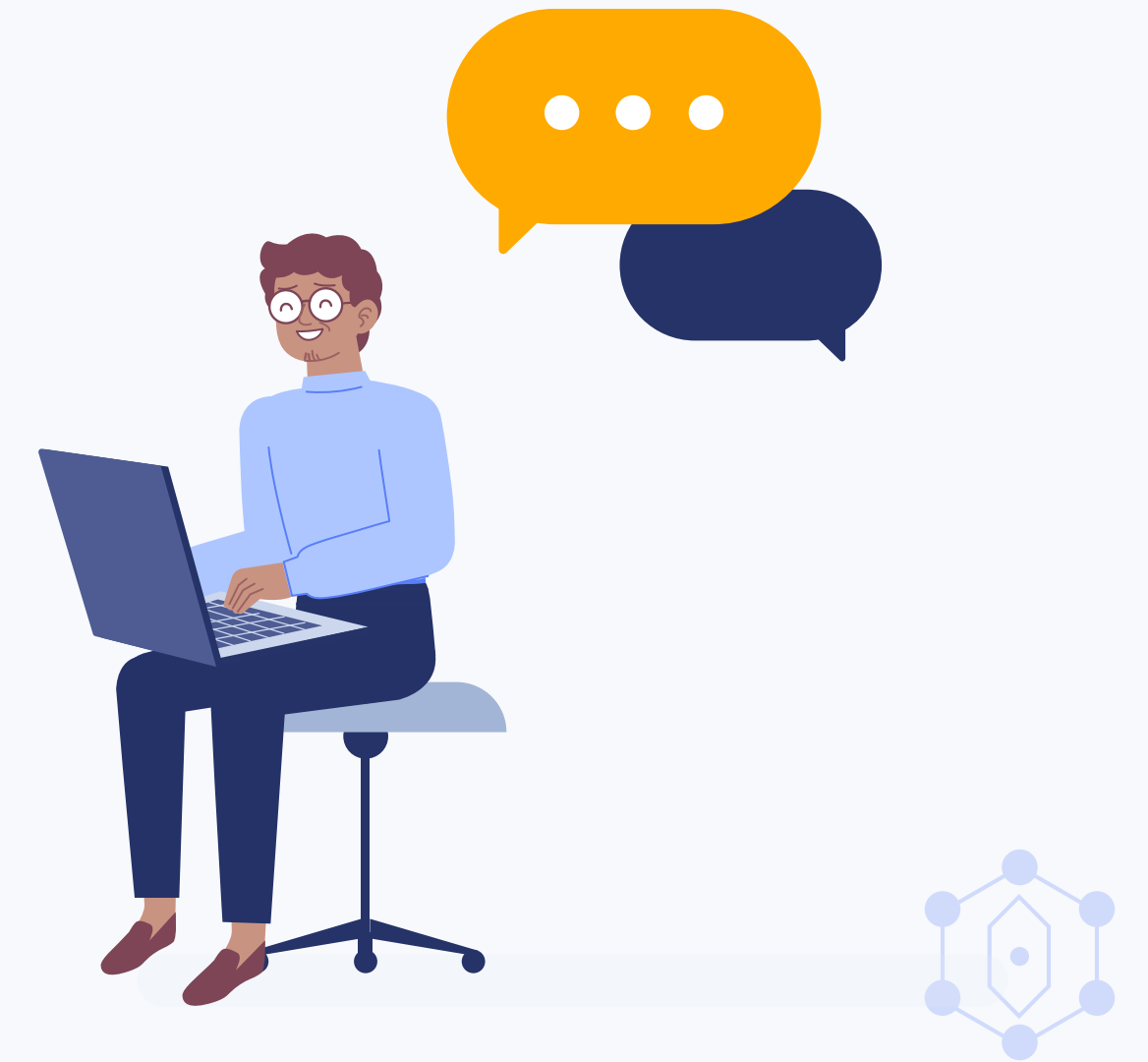
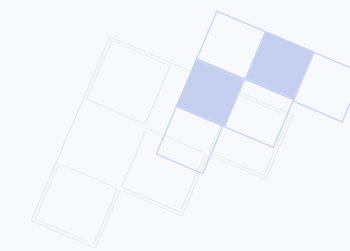
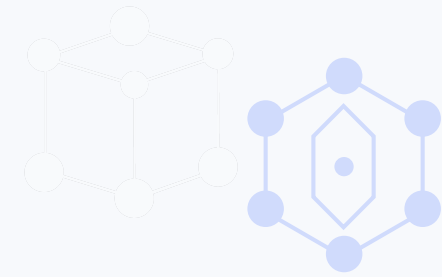
Docente: Jhonatan Andres Mideros Narvaez

Semestre: Cuarto



Recordemos

Conceptos vistos en la anterior clase

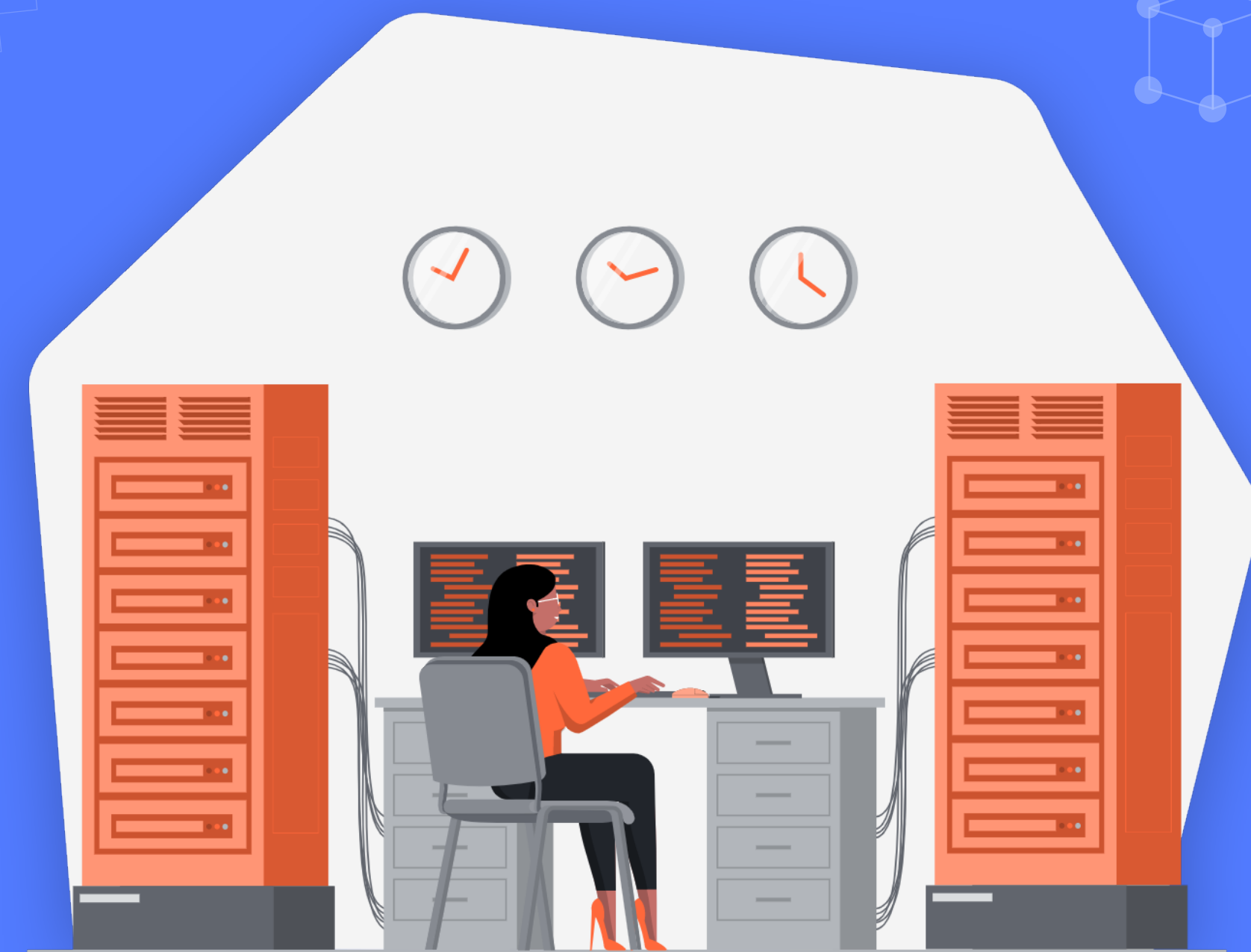


**Patrones
Creacionales**

**Patrones
Estructurales**

**Patrones de
Comportamiento**

Introducción a la POO





Definiciones



La **programación orientada a objetos** es un paradigma de programación que se basa en la idea de **organizar** el código en “**objetos**”.

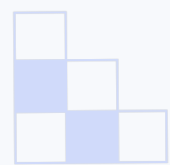
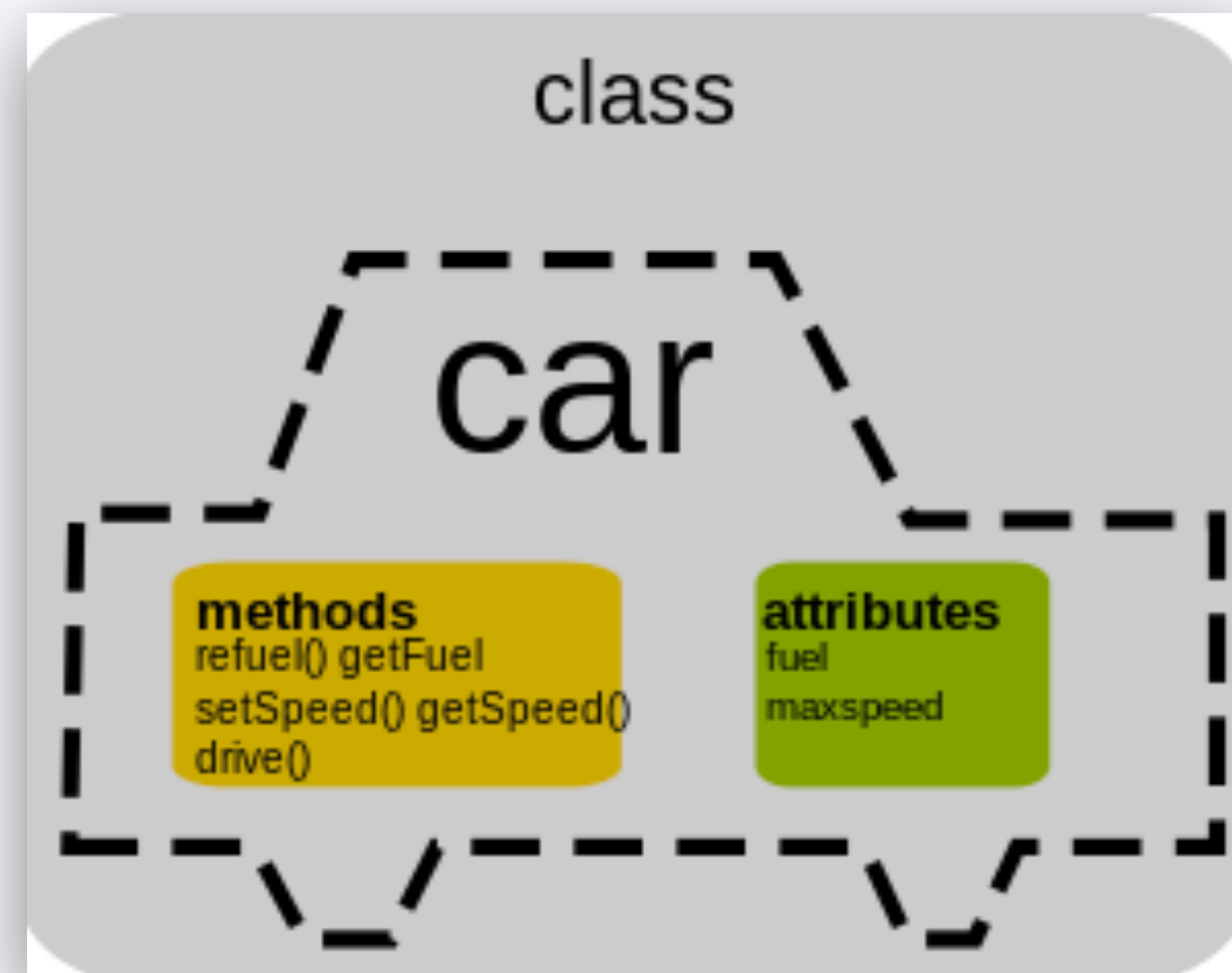


Cada **objeto** tiene **atributos** (datos) y **métodos** (funciones) que operan sobre esos datos.



POO: Clases

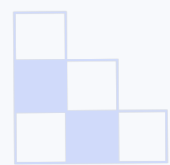
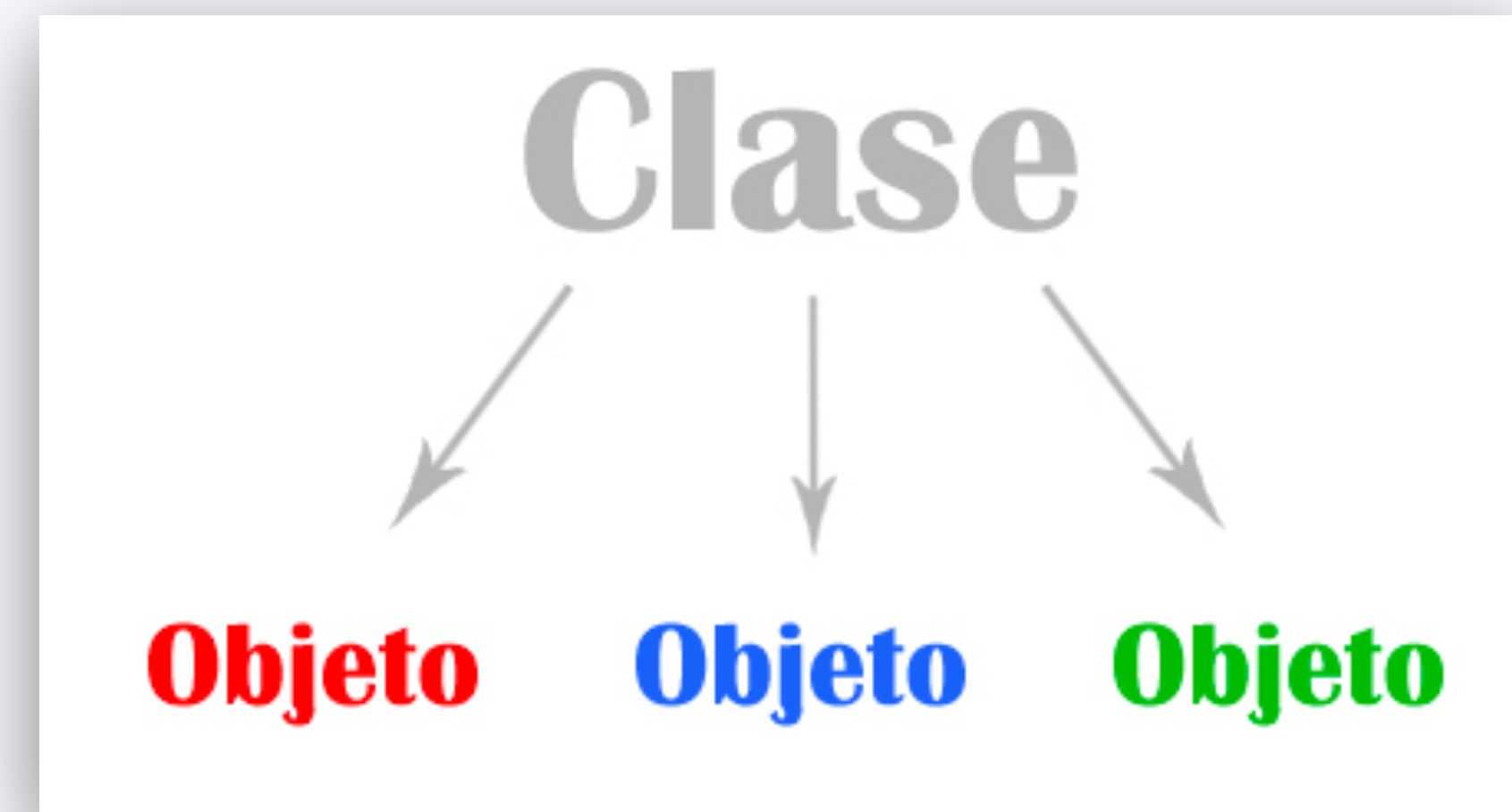
- ✓ Una clase es una **plantilla** o un "molde" que define la **estructura** y el **comportamiento** de los **objetos**. Define atributos (variables) y métodos (funciones) que los objetos de esa clase tendrán.





POO: Objetos

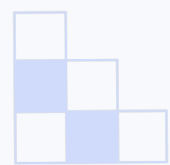
- ✓ Un objeto es una **instancia** concreta de una clase. Contiene datos (**atributos**) y puede realizar acciones (**métodos**).





POO: Encapsulamiento

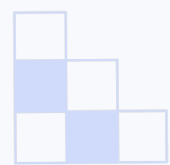
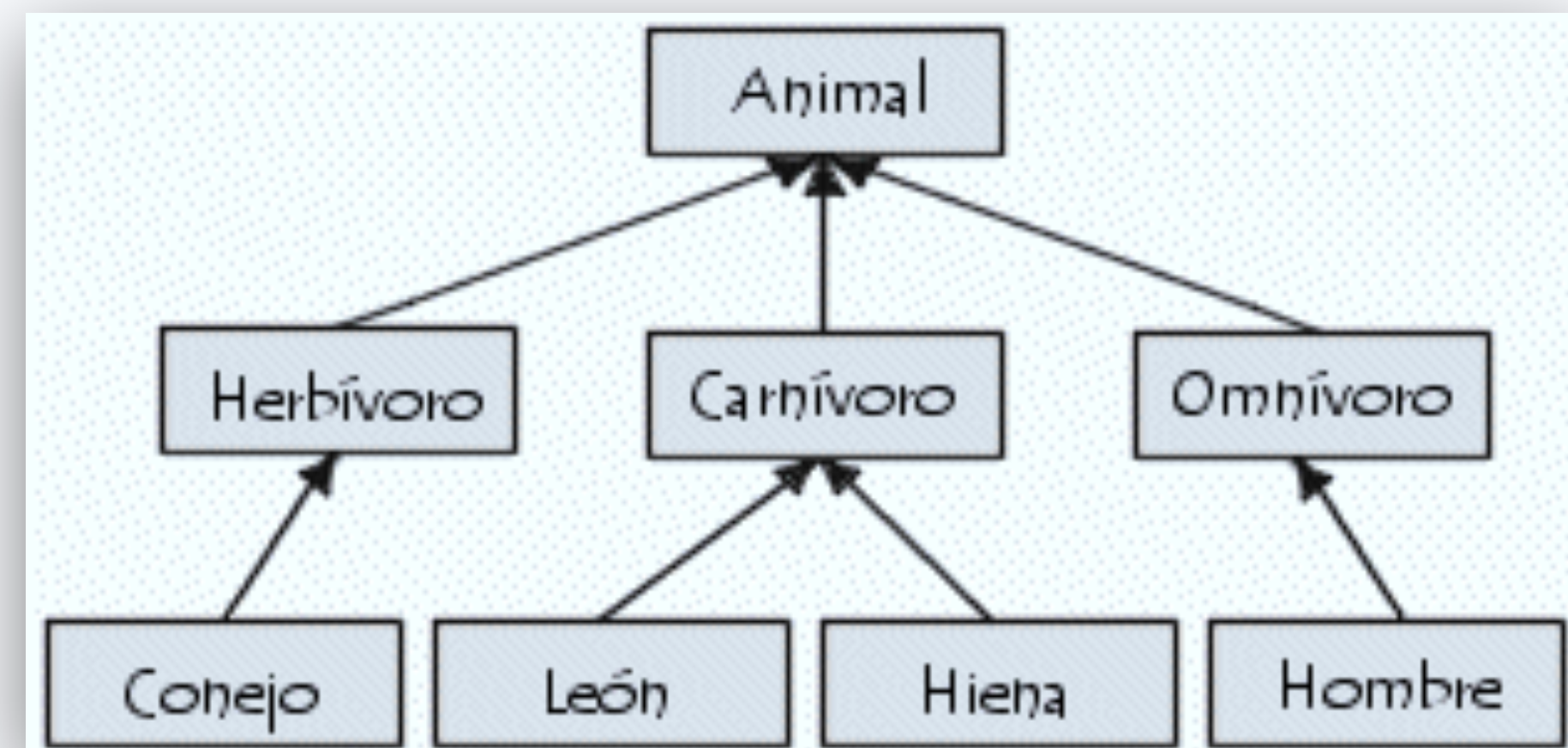
- ✓ Es el principio de **ocultar** los detalles internos de implementación de un objeto y proporcionar una interfaz clara para interactuar con él. Se logra mediante el uso de **modificadores** de acceso (**public**, **private**, **protected**) y métodos **getter** y **setter**.





POO: Herencia

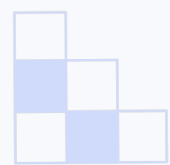
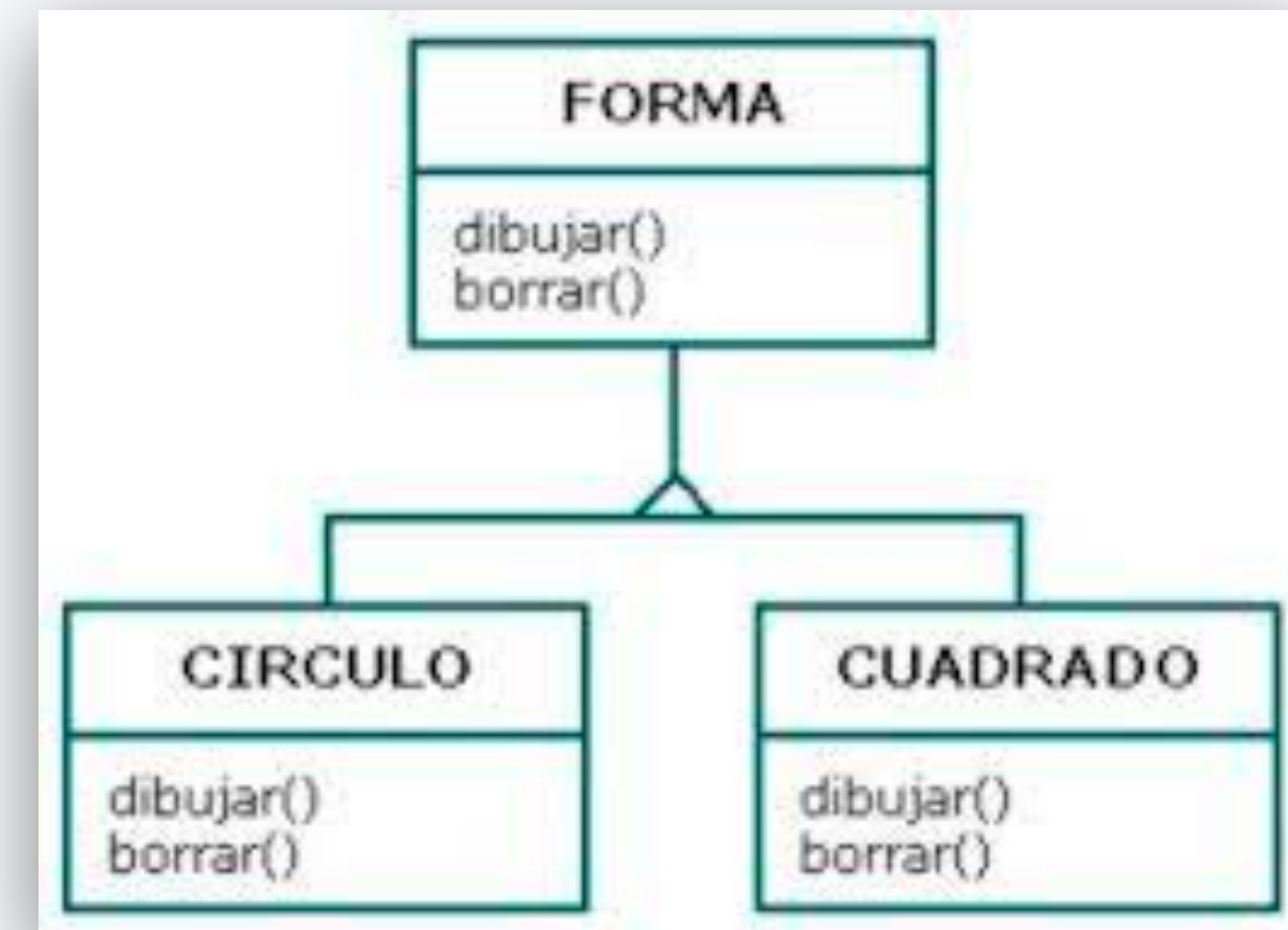
- ✓ Permite que una clase **herede** atributos y métodos de otra clase. Es útil para **reutilizar** código y crear una jerarquía de clases.





POO: Polimorfismo

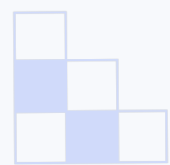
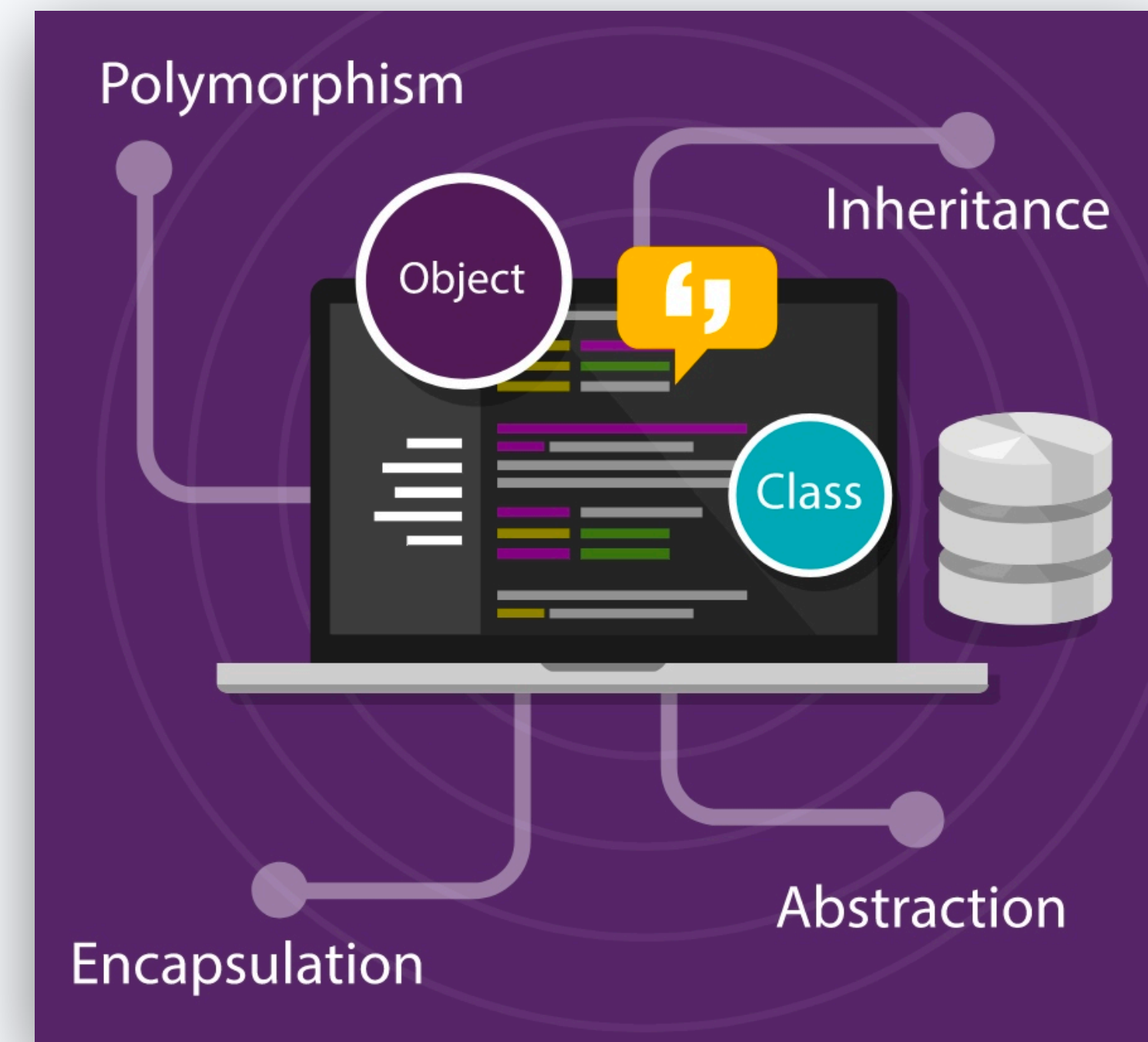
- ✓ Significa "**muchas formas**". Permite que un objeto de una clase pueda **comportarse** como un objeto de otra clase en función del contexto. Se logra a través de la **herencia** y la implementación de **interfaces**.





POO: abstracción

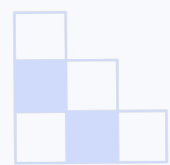
- ✓ En la abstracción, creamos clases que **encapsulan** los **atributos** (datos) y **métodos** (comportamientos) esenciales de un objeto, mientras que ocultamos los detalles internos. Por ejemplo, si estamos modelando un automóvil, podríamos abstraer solo los atributos y métodos relevantes, como la velocidad.

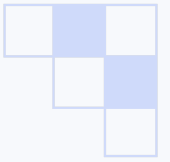




POO: interfaces

✓ Es un **conjunto** de métodos que define un contrato que una clase debe cumplir. En otras palabras, es una **lista de métodos** que una clase debe implementar. Las interfaces permiten establecer un **contrato** entre distintas partes del código sin preocuparse por los detalles de implementación.



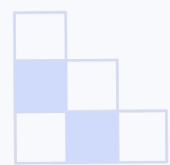


Estructura de una Interfaz

```
// Definición de una interfaz
interface Vehiculo {
    void acelerar();
    void frenar();
}

// Clase que implementa la interfaz Vehiculo
class Automovil implements Vehiculo {
    @Override
    public void acelerar() {
        // Implementación de la aceleración
    }

    @Override
    public void frenar() {
        // Implementación del frenado
    }
}
```



Practica POO



Online-java

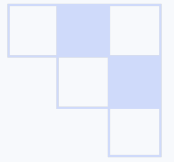


<https://www.online-java.com/>

A screenshot of the Online Java IDE interface. The title bar reads "</> ONLINE JAVA BETA". Below the title bar is a toolbar with icons for file operations (folder, save, share) and settings (gear, info, settings). The main editor area shows a file named "Main.java" with a plus sign to its right. The code is as follows:

```
1
2 /*
3 Online Java - IDE, Code Editor, Compiler
4
5 Online Java is a quick and easy tool that helps you to build, compile, test your programs online.
6 */
7
8 public class Main
9 {
10     public static void main(String[] args) {
11         System.out.println("Welcome to Online IDE!! Happy Coding :)");
12     }
13 }
14
```

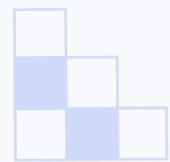
The status bar at the bottom indicates "Ln: 14, Col: 1".



Ejemplo de POO

```
public class Student {  
    // Attributes  
    private String name;  
    private int age;  
  
    // Constructor  
    public Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Methods  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void displayInfo() {  
        System.out.println("Nombre: " + name);  
        System.out.println("Edad: " + age);  
    }  
}
```

```
public class Main  
{  
    public static void main(String[] args) {  
        // Create objects of the class Student  
        Student student1 = new Student("Juan", 20);  
        Student student2 = new Student("María", 22);  
  
        // Accessing object methods and attributes  
        student1.displayInfo();  
        student2.displayInfo();  
    }  
}
```



Patrón Singleton





Definiciones



El patrón Singleton es un patrón de diseño **creacional** que garantiza que una clase tenga **una única instancia** y proporciona un punto de **acceso global** a esa instancia.

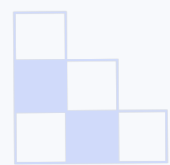


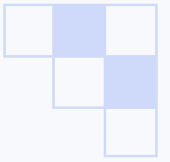
Este patrón es útil cuando deseas asegurarte **de que solo haya una instancia de una clase en toda tu aplicación**, como en casos de configuración global, conexiones a bases de datos, registros, entre otros.



El patrón **Singleton** se compone de los siguientes elementos:

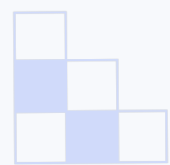
- ✓ **Constructor privado:** Para evitar que otras clases instancien directamente la clase Singleton, su constructor debe ser privado.
- ✓ **Método estático de obtención de instancia:** Se proporciona un método estático que devuelve la única instancia de la clase. Si la instancia aún no existe, se crea; si ya existe, se devuelve la instancia existente.
- ✓ **Instancia única:** La instancia de la clase Singleton debe ser almacenada en una variable estática para que pueda ser compartida por todas las llamadas al método de obtención.





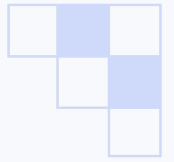
Ejemplo de una clase Singleton

```
public class Singleton {  
    private static Singleton instance;  
  
    private Singleton() {  
        // Constructor private  
    }  
  
    // Method static  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```



Practica del Patron Singleton





Ejemplo Logger con Singleton

```
import java.util.List;
import java.util.ArrayList;

public class Logger {
    private static Logger instance;
    private List<String> logMessages = new ArrayList<>();

    private Logger() {
        // Constructor private
    }

    // Method static
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

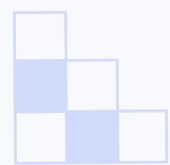
    // Add logs
    public void addLogMessage(String message) {
        logMessages.add(message);
    }

    // Print logs
    public void printLogs() {
        for (String message : logMessages) {
            System.out.println(message);
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Logger logger = Logger.getInstance();
        logger.addLogMessage("Mensaje 1");
        logger.addLogMessage("Mensaje 2");

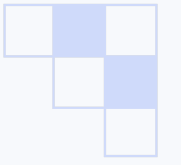
        Logger sameLogger = Logger.getInstance();
        sameLogger.addLogMessage("Mensaje 3");

        logger.printLogs();
    }
}
```



Taller sobre Patron Singleton





Taller de **Patron Singleton** aplicado en un caso de estudio real

- ✔ Seleccione un **caso de estudio** de la **vida real** donde pueda aplicar el patrón **Singleton**. El script debe ser realizado en **java**.

